

25 Fall CSA HW8/Project 3

Team Member	GWID	Task assignment
Akshata Hawanje Shenoy	G40507216	Task 1, Task 2 and Task 5
Eliana Kang	G25866679	Task 3, Task 4 and Task 6

Name	References for <u>Task 1 to Task 4</u>	References for <u>Task 5 and Task 6</u>	Comments
FFT	100,000	7,451,717	Parallel application that simulates the fluid dynamics with FFT
Simple	100,000	27,030,092	Parallel version of the SIMPLE application
Speech	100,000	11,771,664	Kirk Johnson and David Kranz (both at MIT) are responsible for this trace
Weather	100,000	31,764,036	Parallel version of the WEATHER application, which is used for weather forecasting. The serial version is from NASA Space Flight Center, Greenbelt, Md.

Task1. Influence of the Cache Size on the Miss Rate

Configuration

Processors in SMP = 4.

Cache coherence protocol = MESI.

Scheme for bus arbitration = LRU.

Word wide (bits) = 16.

Words by block = 32 (block size = 64 bytes).

Blocks in main memory = 524288 (main memory size = 32 MB).

Mapping = Set-Associative.

Cache sets = They will vary depending on the number of blocks in cache, but you must always have four-way set associative caches (remember: $\text{Number_of_ways} = \text{Number_of_blocks_in_cache} / \text{Number_of_cache_sets}$).

Replacement policy = LRU.

Results

	Miss Rate %			
Blocks in cache	FFT	Simple	Speech	Weather
16	9.062%	57.283%	81.532%	43.943%
32	9.062%	57.283%	81.532%	43.943%
64	8.563%	57.128%	80.806%	37.796%
128	8.789%	51.213%	76.691%	31.130%
256	8.802%	39.133%	73.460%	29.305%
512	8.802%	39.133%	71.332%	29.416%
1024	8.796%	39.133%	70.710%	26.763%
2048	8.802%	39.133%	70.710%	26.669%

Q&A

Q1: Does the global miss rate increase or decrease as the cache size increases? Why? Does this increment or decrement happen for all the benchmarks or does it depend on the different locality grades? What does it happen with the capacity and conflict(collision) misses when you enlarge the caches? And, what does it happen with the compulsory and coherence misses when you enlarge the caches? Are there conflict misses in these experiments? Why?

A1: The global bus traffic decreases as cache size increases. Data traffic drops because larger caches reduce misses and therefore fewer memory blocks are transferred. Address/command traffic drops because fewer misses generate fewer coherence requests, invalidations, and bus transactions. This reduction is not uniform across all benchmarks: programs with small working sets like FFT show minimal change, while programs with larger or shared working sets like Simple, Speech, and Weather experience significant reductions in bus traffic as the cache grows.

Q2: In these experiments, it may be observed that for great cache sizes, the miss rate is stabilized. Why? We can also see great differences of miss rate for a concrete increment of cache size. What do these great differences indicate? Do these great differences of miss rate appear at the same point for all the programs? Why?

A2: The miss rate stabilizes at large cache sizes because the working set of each program eventually fits in the cache, so further increases no longer reduce capacity or conflict misses. The large drops in miss rate at certain cache sizes indicate that the cache has crossed a working-set threshold, eliminating a significant portion of capacity or conflict misses. These drops do not occur at the same cache size for all programs because each benchmark has a different locality pattern and working-set size—so each one benefits at different cache sizes depending on how its memory behaviour interacts with the cache.

Q3: Compare these results with the results obtained in the Task 2 of Project 1. You can observe that the miss rates are higher for multiprocessor traces than for uniprocessor traces. Do you think that, in general, parallel programs exhibit more or less spatial and temporal locality than serial programs? Why? Is it due to the shared data?

A3: Parallel programs in multiprocessor systems generally show higher miss rates than the uniprocessor results from Task 1 because they exhibit weaker spatial and temporal locality, mainly due to shared data. When multiple processors access shared variables or shared data structures, they generate coherence misses, invalidations, and additional evictions that do not occur in a single-core trace. As a result, data that would stay in the cache in the uniprocessor case is frequently replaced or invalidated in the multiprocessor environment. This interference reduces reuse, disrupts locality, and increases overall miss rates. Thus, the higher miss rates observed in these experiments are expected and largely driven by cache coherence activity and shared-data interactions.

Q4: In conclusion, does the increase of cache size improve the multiprocessor system performance?

A4: Yes, increasing the cache size generally improves multiprocessor system performance by reducing capacity and conflict misses, which lowers memory accesses and bus traffic. Programs with larger working sets benefit more, while small-working-set programs (like FFT) see little change. Compulsory and coherence misses remain largely unaffected. Overall, larger caches help performance, but gains diminish once the working set fits, and coherence-related overheads still limit improvement.

Task2. Influence of the Cache Size on the Bus Traffic

Configuration

Processors in SMP = 4.

Cache coherence protocol = MESI.

Scheme for bus arbitration = LRU.

Word wide (bits) = 16.

Words by block = 32 (block size = 64 bytes).

Blocks in main memory = 524288 (main memory size = 32 MB).

Mapping = Set-Associative.

Cache sets = They will vary depending on the number of blocks in cache, but you must always have four-way set associative caches (remember: $\text{Number_of_ways} = \text{Number_of_blocks_in_cache} / \text{Number_of_cache_sets}$).

Replacement policy = LRU.

Results

	Block transfers Multiply by 64 to get data traffic			
Blocks in cache	FFT	Simple	Speech	Weather
16	$10342 \times 64 = 61888$	$64342 \times 64 = 117888$	$104799 \times 64 = 6707136$	$47342 \times 64 = 029888$
32	$10342 \times 64 = 61888$	$64342 \times 64 = 117888$	$104799 \times 64 = 6707136$	$47342 \times 64 = 029888$
64	$9830 \times 64 = 629120$	$64316 \times 64 = 1,16,224$	$104124 \times 64 = 6663936$	$40666 \times 64 = 602624$
128	$9901 \times 64 = 633664$	$57341 \times 64 = 6,69,824$	$98464 \times 64 = 301696$	$33610 \times 64 = 151040$
256	$9941 \times 64 = 636224$	$44752 \times 64 = 8,64,128$	$94483 \times 64 = 046912$	$31709 \times 64 = 029376$
512	$9941 \times 64 = 636224$	$44752 \times 64 = 8,64,128$	$91790 \times 64 = 874560$	$31662 \times 64 = 026368$
1024	$9941 \times 64 = 636224$	$44752 \times 64 = 8,64,128$	$91111 \times 64 = 5831104$	$29074 \times 64 = 860736$
2048	$9941 \times 64 = 636224$	$44752 \times 64 = 8,64,128$	$91060 \times 64 = 5827840$	$28996 \times 64 = 855744$

	Bus transactions Multiply by 6 to get address traffic			
Blocks in cache	FFT	Simple	Speech	Weather
16	12453*6=7 4718	70786*6=42 4716	105016*6=6 30096	49515*6=2 97090
32	12453*6=7 4718	70786*6=42 4716	105016*6=6 30096	49515*6=2 97090
64	12077*6=7 2462	70635*6=4, 23,810	104525*6=6 27150	43094*6=2 58564
128	11974*6=7 1844	63510*6=3, 81,060	99188*6=59 5128	36423*6=2 18538
256	12060*6=7 2360	51176*6=30 7056	95130*6=57 0780	34580*6=2 07480
512	12060*6=7 2360	51176*6=30 7056	92625*6=55 5750	34475*6=2 06850
1024	12060*6=7 2360	51176*6=30 7056	91896*6=55 1376	31851*6=1 91106
2048	12060*6= 72360	51176*6=30 7056	91842*6=55 1052	31650*6=1 89900

	Bus Traffic per memory access (in bytes)			
	(Address Traffic + Data Traffic)/References			
Blocks in cache	FFT	Simple	Speech	Weather
16	1.36606	45.42604	73.37232	33.26978
32	1.36606	45.42604	73.37232	33.26978
64	7.01582	45.40034	72.91086	28.61188
128	7.05508	40.50884	68.96824	23.69578
256	7.08584	31.71184	66.17692	22.36226
512	7.08584	31.71184	64.3031	22.33218
1024	7.08584	31.71184	63.8248	20.51842
2048	7.08584	31.71184	63.78892	20.45644

Q&A

Q1: Does the global bus traffic increase or decrease as the cache size increases? Why (give two reasons, one for the data traffic and another for the address+command bus traffic)? Does this increment or decrement happen for all the benchmarks?

A1: The global bus traffic decreases as cache size increases. Data traffic decreases because larger caches retain more frequently accessed blocks, reducing memory fetches. Address and command traffic decreases because fewer cache misses lead to fewer coherence transactions on the bus. This trend is not uniform for all benchmarks—programs with poor locality or large working sets (e.g., Speech) still generate significant bus traffic even at large cache sizes, while programs like FFT show minimal bus traffic throughout.

Q2: In these experiments, it may be observed that for great cache sizes, the bus traffic is stabilized. Why? We can also see great differences of bus traffic for a concrete increment of cache size. What do these great differences indicate? Do these great differences of bus traffic appear at the same point for all the programs? Why?

A2: For large cache sizes, the bus traffic stabilizes because the programs' working sets fit in the cache, so additional cache blocks no longer reduce memory accesses or coherence transactions. The large drops in bus traffic at certain cache sizes indicate that the cache has crossed a working-set threshold, eliminating many capacity or conflict misses and reducing coherence activity. These drops occur at different points for different programs because each benchmark has a unique working set and memory-access pattern, so the cache begins fitting key data at different sizes.

Q3: In conclusion, does the increase of cache size improve the multiprocessor system performance? Are the conclusions you obtain similar to the previous ones for the miss rate (Task 1)?

A3: Yes, increasing the cache size improves multiprocessor system performance by reducing both the miss rate (Task 1) and the bus traffic (Task 2). Larger caches hold more of the working set, which decreases capacity and conflict misses, reduces memory accesses, and lowers coherence-related bus transactions. These improvements are more pronounced for programs with larger working sets or lower locality, while small-working-set programs benefit less. Overall, the conclusions from Task 2 are consistent with Task 1, showing that larger caches enhance performance by improving hit rates and reducing bus traffic.

Task 3. Influence of the Cache Coherence Protocol on the Miss Rate

Configuration

Processors in SMP = 4.

Scheme for bus arbitration = LRU.

Word wide (bits) = 16.

Words by block = 32 (block size = 64 bytes).

Blocks in main memory = 524288 (main memory size = 32 MB).

Blocks in cache = 256 (cache size = 16 KB).

Mapping = Set-Associative.

Cache sets = 64 (four-way set associative caches).

Replacement policy = LRU.

Results

	Global miss rate			
Cache Coherence Protocol	FFT	Simple	Speech	Weather
MSI	8.718%	39.123%	73.458%	29.638%
MESI	8.802%	39.133%	73.460%	29.305%
DRAGON	0.424%	29.496%	73.401%	22.627%

Q&A

Q1: Do all the protocols have the same miss rate? Which is the coherence protocol with the best miss rate? And which does it have the worst? In particular, is the miss rate the same for the MSI and MESI protocols? Why?

A1: No, the protocols have different miss rates. Overall, the coherence protocol with the best miss rate is DRAGON. The coherence protocol with the worst overall miss rate is MESI. The miss rate for the MSI and MESI protocols are similar (with MSI having better overall miss rate) but not the same. Surprisingly, the MSI protocol performs slightly better than MESI protocol, meaning that the exclusive state of the MESI protocol is negligible to improvement because there are high amounts of data sharing between processors.

Q2: Do you observe any difference between the update-based protocol and the invalidation based protocols? Which? Why? Are the coherence misses the same for these two kinds of protocols?

A2: Yes, the update-based protocol (DRAGON) has better performance than the invalidation protocols (MSI & MESI). The DRAGON protocol performs better because a cache update is sent to all processors so the old value doesn't cause an invalidation.

Q3: Do you think that the results and conclusions obtained with these experiments are of general application or they may change depending on the used benchmarks?

A3: The results and conclusions vary depending on the application or benchmark. For example, if an application or benchmark requires more reading of invalidated data, the invalidation protocols would have higher cache misses and more bus traffic. If few reads of invalidated data is required, then the cache misses of the invalidation protocols would be better.

Q4: In conclusion, does the use of a concrete cache coherence protocol improve the multiprocessor system performance? Why?

A4: Yes, the use of a cache coherence protocol improves performance because it allows efficient communication between processors to maintain data consistency. Without these protocols, there would be no way to update old values, resulting in higher misses and reduced performance.

Task 4. Influence of the Cache Coherence Protocol on the Bus Traffic

Configuration

Processors in SMP = 4.

Scheme for bus arbitration = LRU.

Word wide (bits) = 16.

Words by block = 32 (block size = 64 bytes).

Blocks in main memory = 524288 (main memory size = 32 MB).

Blocks in cache = 256 (cache size = 16 KB).

Mapping = Set-Associative.

Cache sets = 64 (four-way set associative caches).

Replacement policy = LRU.

Address traffic = Bus transactions x 6

Data traffic = Block transfers x 64

References = 100,000

Results

	Bus Traffic per memory access (in bytes)			
	(Address Traffic + Data Traffic)/References			
Cache Coherence Protocol	FFT	Simple	Speech	Weather
MSI	$\frac{((12083 \times 6) + (9913 \times 64))}{100000}$ 7.0693	$\frac{((51169 \times 6) + (44807 \times 64))}{100000}$ 31.74662	$\frac{((104420 \times 6) + (103546 \times 64))}{100000}$ 72.53464	$\frac{((34853 \times 6) + (31962 \times 64))}{100000}$ 22.54686
MESI	$\frac{((12060 \times 6) + (9941 \times 64))}{100000}$ 7.08584	$\frac{((51176 \times 6) + (44752 \times 64))}{100000}$ 31.71184	$\frac{((95130 \times 6) + (94483 \times 64))}{100000}$ 66.17692	$\frac{((34580 \times 6) + (31709 \times 64))}{100000}$ 22.36856
DRAGON	$\frac{((16680 \times 6) + (987.21875 \times 64))}{100000}$ 1.63262	$\frac{((46215 \times 6) + (33570.87500 \times 64))}{100000}$ 3.10860875	$\frac{((95389 \times 6) + (94482.25 \times 64))}{100000}$ 66.19198	$\frac{((35631 \times 6) + (24849.78125 \times 64))}{100000}$ 18.04172

Q&A

Q1: Do all the protocols have the same bus traffic? Which is the coherence protocol with the best bus traffic? And which does it have the worst? In particular, is the bus traffic the same for the MSI and MESI protocols? Why (for this answer, remember how the miss rate was for these two protocols - Task 3)?

A1: No, although MSI and MESI protocols have similar bus traffic, all three protocols have different values for bus traffic. DRAGON has the best bus traffic, and MSI has the worst bus traffic. The MSI and MESI protocols have similar bus traffic because their miss rates were also similar in task 3. The similar miss rates result in both protocols having similar address traffic and data traffic, as they have similar number of read misses and write misses.

Q2: Do you observe any difference between the update-based protocol and the invalidation-based protocols? Which? Why (give at least two reasons)?

A2: The DRAGON protocol has the best performance for FFT, Simple, and Weather, while it has the second-worst performance for Speech. DRAGON performs the best for the three traces because the traces may have a high read-write sharing between processors, where the same data is read frequently. DRAGON performs the second-worst for Speech because data may not be read by other processors, meaning that data may be broadcast even if other processors don't need that data, resulting in worse performance due to a congested bus.

Q3: Do you think that the results and conclusions obtained with these experiments are of general application, or may they change depending on the used benchmarks? Indicate another scenario in which the invalidation protocol does much better than the update protocol.

A3: The results and conclusions obtained vary depending on the benchmark. In a scenario where the invalidation protocols perform better than the update protocol, the benchmark would require fewer reads and less use for updating the value for all processors, like the Speech trace.

Q4: In conclusion, does the use of a concrete cache coherence protocol improve the multiprocessor system performance? Why? Are the conclusions you obtain similar to the previous ones for the miss rate (Task 3)?

A4: Yes, the use of a cache coherence protocol improves the multiprocessor performance because using the right protocol, as we observed above, determines how data will be updated to improve misses and performance. Therefore, the conclusions obtained are similar to task 3, but just that the efficiency of a protocol depends on the benchmark.

Task 5. Influence of the Number of Processors on the Miss Rate

Configuration

Cache coherence protocol = MESI.

Scheme for bus arbitration = LRU.

Word wide (bits) = 16.

Words by block = 32 (block size = 64 bytes).

Blocks in main memory = 524288 (main memory size = 32 MB).

Blocks in cache = 256 (cache size = 16 KB).

Mapping = Set-Associative.

Cache sets = 64 (four-way set associative caches).

Replacement policy = LRU.

Results

Number of Processors	FFT	Simple	Weather
1	8.12%	14.64%	15.21%
2	20.95%	23.97%	20.09%
4	26.87%	30.97%	22.46%
8	32.53%	36.35%	26.93%

Q&A

Q1: Does the global miss rate increase or decrease as the number of processors increases? Why?

A1: The global miss rate increases as the number of processors increases. This happens because more processors share and modify data, causing frequent coherence actions under MESI that invalidate cached lines. As a result, temporal locality decreases, and each processor experiences more coherence misses. Additionally, inter-processor interference increases conflict and capacity pressure in the cache, further raising the miss rate. This trend is clear in the results: for example, FFT rises from 8.12% (1 processor) to 32.53% (8 processors).

Q2: Does this increment or decrement happen for all the benchmarks or does it depend on the different locality grades, shared data...?

A2: The increase in miss rate with more processors does not happen uniformly for all benchmarks; it depends on locality and shared data patterns. Programs with more shared or frequently modified data (e.g., Simple and Weather) see larger increases due to coherence misses, while programs with mostly private or well-localized data (e.g., FFT) show smaller increases. Thus, the effect is workload-dependent, influenced by both temporal/spatial locality and the degree of inter-processor sharing.

Q3: What does it happen with the capacity and coherence misses when you enlarge the number of processors?

A3: When the number of processors increases, capacity misses may increase because each processor competes for a limited cache size, effectively reducing the cache available per thread. Coherence misses also increase since more processors generate more invalidations and remote fetches under the coherence protocol. Together, these effects cause the overall miss rate to rise as the processor count grows.

Q4: Are there conflict misses in these experiments? Why?

A4: Yes, conflict misses are present in these experiments. Since the caches are set-associative with limited ways, multiple addresses can map to the same set and evict each other, especially as more processors access the cache simultaneously. Increasing the number of processors intensifies this effect, contributing to higher conflict misses along with capacity and coherence misses.

Q5: Do you think that the results and conclusions obtained with these experiments are of general application or they may change depending on the cache coherence protocol? Why?

A5: The results and conclusions may change depending on the cache coherence protocol. Different protocols handle shared data and invalidations differently, which directly affects coherence misses and overall miss rates. For example, DRAGON reduces invalidations and preserves shared lines longer, resulting in lower miss rates than MESI or MSI. Therefore, while trends like miss rate increasing with more processors are general, the magnitude and exact behavior depend on the chosen coherence protocol.

Q6: In conclusion, does the increase of the number of processors improve the multiprocessor system performance? Why and in what sense?

A6: Increasing the number of processors does not necessarily improve multiprocessor performance in terms of cache efficiency. While more processors increase computational throughput, they also raise coherence and capacity misses, reducing cache hit rates and increasing memory traffic. Therefore, performance gains are limited by memory contention and

inter-processor interference, and the system may become less efficient per processor as the processor count grows.

Task 6. Influence of the Number of Processors on the Bus Traffic

Configuration

Cache coherence protocol = MESI.

Scheme for bus arbitration = LRU.

Word wide (bits) = 16.

Words by block = 32 (block size = 64 bytes).

Blocks in main memory = 524288 (main memory size = 32 MB).

Blocks in cache = 256 (cache size = 16 KB).

Mapping = Set-Associative.

Cache sets = 64 (four-way set associative caches).

Replacement policy = LRU.

FFT Reference = 7451717

Simple Reference = 27030092

Speech Reference = 11771664

Weather Reference = 31764036

Results

GIVEN	Block transfers (You need to Multiply by 64 to get data traffic)		
Number of Processors	FFT	Simple	Weather
1	928043	5357756	5914755
2	2234070	8564862	7761028
4	2591185	10117524	8480669
8	2813633	10996624	9646692

GIVEN	Bus transactions (You need to Multiply by 6 to get address traffic)		
Number of Processors	FFT	Simple	Weather
1	928043	5357756	5914755
2	2629642	9674938	8662469
4	3051536	11539535	9463257
8	3283223	12503960	10859530

YOU NEED TO DO	Bus Traffic per memory access (in bytes) (Address Traffic + Data Traffic)/References in Table 2		
Number of Processors	FFT	Simple	Weather
1	8.717857911	13.87501456	13.03464239
2	21.30493308	22.42688615	17.27364262
4	24.71176187	26.51706646	18.87487969
8	26.8088348	28.81261729	21.48799567

Q&A

Q1: Does the global bus traffic increase or decrease as the number of processors increases? Why (give two reasons, one for the data traffic and another for the address+command bus traffic)? Does this increment or decrement happen for all the benchmarks, or does it depend on the different locality grades, shared data, ...?

A1: As the number of processors increases, the global bus traffic increases for all benchmarks. The reason being: 1) The number of block transfers increases for all benchmarks as processors increase, and 2) The address+command bus traffic increases for all benchmarks as processors increase. This means that more cores share data, which causes more invalidations, read misses, and block transfers. In addition, there will be more snooping traffic. However, the trend depends on shared data, locality, and synchronization

Q2: In general, which is the step with more bus traffic: from 1 to 2 processors, from 2 to 4 processors, or from 4 to 8 processors? Why?

A2: The bus traffic is higher in the jump from 1 processor to 2 because there is no traffic at 1 processor, but the introduction of another processor increased bus traffic.

Q3: Do you think that the results and conclusions obtained with these experiments are of general application, or may they change depending on the cache coherence protocol? Why?

A3: The results and conclusions may change depending on the cache coherence protocol since each protocol generates different bus traffic patterns, but the general trend of traffic increasing with increasing processors remains true.

Q4: Change the results obtained with these experiments in order to measure the bus traffic in bytes per instruction (you must divide by the number of instruction captures -not memory accesses- in the traces). Supposing that these applications run at a sustained 100 MIPS per processor (and they are integer-intensive applications), what is the address and data bus bandwidth requirement for each application and configuration of the number of processors?

A4:

FFT instruction capture = 3124454; Simple instruction capture = 11594172; Weather instruction capture = 13638207

Address Bus Bandwidth = Number of Processors $\times$ 100MIPS $\times$ Address Traffic per Instruction

Data Bus Bandwidth = Number of Processors $\times$ 100MIPS $\times$ Data Traffic per Instruction

Address bus bandwidth (in Mb/s)			
processor	FFT	SIMPLE	WEATHER
1	$= (100 \times 928043 \times 6) / 3124454$ 178.2153938	277.265	260.214
2	1009.96	1001.36	762.194
4	2343.99	2388.69	1665.31
8	5043.91	5176.65	3822.04

Data bus bandwidth (in MB/s)			
processor	FFT	SIMPLE	WEATHER
1	$= (100 \times 928043 \times 64) / 3124454$ 1900.96	2957.49	2775.62
2	9152.35	9455.63	7284.03

4	21230.7	22339.6	15918.9
8	46106.6	48561.2	36215.2

Q5: The computation in the question above gives the average bandwidth requirement under the assumption that the bus bandwidth is enough to allow the processors to execute at full speed. This computation provides a useful basis for sizing the system. For example, on a machine with 4.8 GB/s of data bandwidth, how many processors will the bus be able to support without saturating for each benchmark? Remember: If the bandwidth is not sufficient to support the application, the application will slow down.

A5: Processors = Available bandwidth/Bandwidth per processor

4.8 GB/s = 4800 MB/s

P FFT: $4800/1900.96 = 2.525$; $4800/9152.35 = 0.524$; $4800/21230.7 = 0.226$; $4800/46106.6 = 0.104$

P Simple: $4800/2957.49 = 1.623$; $4800/9455.63 = 0.507$; $4800/22339.6 = 0.214$; $4800/48561.2 = 0.098$

P Weather: $4800/2775.62 = 1.729$; $4800/7284.03 = 0.659$; $4800/15918.7 = 0.302$; $4800/36215.2 = 0.132$

On a machine with 4.8 GB/s of data bandwidth, FFT can have 2 processors, Simple can have 1 processor, and Weather can have 1 processor.

Q6: In conclusion, does the increase in the number of processors improve the multiprocessor system performance? Why and in what sense? Are the conclusions you obtain similar to the previous ones for the miss rate (Task 5)?

A6: Yes, the increase in the number of processors improves the performance by allowing the bus not to be saturated, and the bandwidth is sufficient. If the bus is already saturated/bottlenecked, as in question 5, adding more processors won't improve the performance because the system will stall waiting for the bus. Similar to task 5, adding more processors increases cache misses due to more sharing, but for this task, the limit is the bandwidth. Hence, the miss rate and bus traffic show that more processors are beneficial only if there are sufficient system resources to support them.